

Al ahaMatic — Technology Decision Sheet

Ordered by how expensive each decision is to undo — not by how much debate it generated.

Prepared for: Project Lead

Status: all items provisional — nothing below is final until you approve it

2026-07-29

How to read this. Every item below is a recommendation, not a done deal — nothing takes effect until you sign off. The six decisions are ordered from **hardest to undo later** (the database) to **easiest to undo later** (which programming language and web framework we use). That ordering is deliberate: the language and framework question is the one that generated the most back-and-forth in review, and it turns out to be the cheapest of the six to change your mind about later. The database and data-sync questions generated the least discussion so far, and they are the two that would be the most painful to reverse once real customer data is sitting in them. This sheet is arranged so your attention lands where the cost of being wrong is highest.

Brutal	Very High	High	Moderate-High	Moderate	Low (cheapest)
--------	-----------	------	---------------	----------	----------------

⚠ The one thing that needs your decision before anything else

We have already picked a database design (below), but there is a second, related question we have not answered: should the system support two-way data syncing — for example, someone editing on a phone while offline, then reconciling with the server later — or should the server always be the single source of truth?

This isn't a side issue. If the answer is ever "yes, support two-way sync," every table in the database needs to be built differently from day one: extra columns to track when a row last changed, "soft delete" markers instead of actually deleting rows, and a specific rule per table for what happens when two edits conflict. Bolting that on after the database is already built and full of customer data is a much bigger job than building it in from the start.

We made the database call (schema-per-tenant, PostgreSQL, the identifier format — all below) before this question was answered. That is out of sequence, and we're flagging it rather than quietly proceeding.

Ask of you: either (a) tell us now whether two-way sync is required, so we can build the database to match, or (b) explicitly accept that the database decision below may still need to change once this is answered.

1. Data model & database	DIVERGES FROM YOUR BRIEF
2. Data-sync posture	UNDECIDED NEW — NOT IN EARLIER REVIEWS
3. System architecture & how systems talk to each other	PARTLY DIVERGES
4. Cloud provider	NEW COMPARISON, AT YOUR REQUEST
5. Web & mobile app technology	WE REVERSED OURSELVES TWICE
6. Programming language & web framework	DISAGREES WITH YOUR BRIEF'S RANKING

1 Data model & database

BRUTAL TO REVERSE

DIVERGES FROM YOUR BRIEF'S DEFAULT

WHAT WE RECOMMEND

PostgreSQL as the main database engine (with MySQL and SQL Server also supported, in case a customer requires them). Each customer ("tenant") gets their own separate schema inside the database — not shared tables with a customer-ID column. Record IDs are a modern sortable-random format (UUIDv7) rather than simple counting numbers (1, 2, 3...).

WHERE THIS DIFFERS FROM YOUR BRIEF

Your brief's default was: one shared set of tables for all customers, with a "which customer does this row belong to" column, protected by the database's built-in row-level security feature. We are recommending against that, for two concrete reasons:

- **Row-level security isn't available the same way everywhere.** Postgres and SQL Server support it, in different ways; MySQL doesn't support it at all. Since we need to support all three, relying on it would tie us to specific databases.
- **Our platform is different from the single business application your brief assumed.** Al ahaMatic isn't one HRIS or finance app — it's a platform other people use to build many different applications, and each of those builders defines their own data structure at runtime, after the platform is already live. A shared-table design assumes everyone's data looks similar; our builders' data won't.

Giving each customer their own schema means one customer's data is walled off structurally, not just by a rule in application code that a bug could accidentally skip — which matters more here than in a single fixed-schema product.

WHY THIS IS THE HARDEST OF THE SIX TO UNDO

This decides the physical shape every customer's data is stored in. Changing it later — say, moving from separate schemas to shared tables, or switching primary database engines — means migrating every customer's live data into a different structure while the platform is running. That's the most expensive kind of change on this list, by a wide margin.

2 Data-sync posture

VERY HIGH TO REVERSE — AND STILL UNDECIDED

UNDECIDED

NEW — THIS QUESTION DIDN'T EXIST IN THE EARLIER STACK REVIEW

THE QUESTION, IN PLAIN TERMS

When someone uses a mobile app built on this platform, and their phone temporarily has no signal, what happens to their edits? Two options: (a) the server is always the single source of truth, and the app simply can't save changes until it's back online; or (b) the app lets you keep working offline, and reconciles your changes with the server later, possibly alongside someone else's changes to the same data.

This is not decided. It surfaced from your brief and the standup discussion, and neither the original stack review nor anything since has settled it.

See the callout at the top of this sheet — this is the same issue. It's listed here in its place in the cost-ordering, and again at the top because it's currently the single blocking question on this whole list.

3 System architecture & how systems talk to each other

HIGH TO REVERSE

PARTLY DIVERGES FROM YOUR BRIEF

WHAT WE RECOMMEND

Build the platform as one deployable piece of software (not dozens of small independent services), but organize the code inside it into clearly separated modules, with automated checks that block anyone (including an AI writing code) from writing code that improperly crosses from one module into another. Separately: external systems will talk to our platform using a widely-adopted, standard, machine-readable format for describing what's available (OpenAPI) — not a more flexible but harder-to-verify format (GraphQL), and not a format that only works if the other system is also written in our exact programming language.

WHERE THIS DIFFERS FROM YOUR BRIEF

Your brief's own default landed on roughly the same "keep it as one deployable thing" answer, but for a different reason — mainly that our old platform (which we are not reusing or copying) was built that way. We reached the same conclusion independently: because a large part of this platform is built by an AI agent, we specifically need boundaries between modules that a machine can automatically verify are being respected on every single change. Splitting into many small independent services would make those boundary violations invisible until they show up as a live bug, which is the opposite of what we need. So: same destination as your brief, different and — we think — stronger reasoning.

On the "how systems talk to each other" question, we are recommending against a more flexible, query-anything format (GraphQL) that your brief didn't rule out. The reason is specific to multi-tenant security: that flexible format makes it much harder to guarantee, automatically and in advance, that one customer's query can never even reveal that another customer's data exists — a guarantee we consider non-negotiable.

WHY THIS IS EXPENSIVE TO UNDO

Splitting a single deployable system into many separate services later — or changing the external "talk to us" format after other companies have integrated against it — is a major undertaking once real integrations exist, though less catastrophic than restructuring the database.

4 Cloud provider

MODERATE-HIGH TO REVERSE

NEW COMPARISON, DONE AT YOUR REQUEST

WHAT WE RECOMMEND

Google Cloud as our default and starting point, while designing so that a future move to AWS or Microsoft Azure would be a real but manageable project rather than a rewrite from scratch.

What we compared	Google Cloud	AWS	Azure
Effort to set up and keep running	Lowest — simplest to deploy and maintain	Highest — more pieces to configure and manage	In between
Database hosting cost	Lowest at our expected size	Mid	Highest
Documentation & examples available (matters for AI-written code)	Good	Largest by a clear margin	Good
Handles extreme scale	Sufficient for our targets	Deepest track record at very large scale	Sufficient for our targets

We are already running on Google Cloud today (having previously moved from a smaller provider), which makes Google Cloud the "no migration cost" choice as well — a real, practical factor, but a business one, not a technical finding, and we've kept it separate from the technical comparison above.

THE ONE THING TO FLAG CLEARLY HERE

Our application code is portable between cloud providers. Our infrastructure setup — servers, networking, permissions — is not. If we ever move providers, that's a genuine infrastructure project that needs its own budget and timeline; it is not something we get "for free" just because we wrote portable application code. We're designing the infrastructure setup itself (using a provider-neutral configuration tool) so that project would be a rewrite of configuration, not a redesign of the whole system — but it's still a project.

5 Web & mobile app technology

MODERATE TO REVERSE

WE REVERSED OUR OWN POSITION TWICE ON THIS ONE

WHAT WE RECOMMEND

Anything built on the platform should be offered as a real web version *and* a genuine mobile app — not just a mobile-friendly website — and the person building it chooses which they want. A "works okay in a mobile browser" website is not enough on its own, because things like appearing in the app stores, and using the phone's camera or location, need a real app.

MOBILE APP TECHNOLOGY

React Native (with a modern toolkit called Expo built on top of it).

FULL HONESTY ABOUT HOW WE GOT HERE — WE CHANGED OUR MIND TWICE

Round 1: we originally recommended React Native, mainly because it uses the same programming language as the rest of the system, which is a real, practical advantage.

Round 2: we then switched our recommendation to Flutter, a different technology, based on claims that its tooling was more self-contained and less prone to breaking over time.

Round 3 (where we landed): we then went back and actually checked those claims against current, up-to-date sources — and two of the three turned out to be wrong or out of date. The specific things we'd claimed about React Native being harder to maintain either no longer apply, or turned out to be closer to true of Flutter's own tooling instead. On checking the facts rather than relying on our earlier assessment, we moved back to React Native.

The lesson we're recording, for your awareness: claims like "this tool is well-maintained" or "that tool is unstable" go out of date faster than anything else we compare, and from now on we check them against current sources before letting them decide anything — rather than relying on what we remembered from an earlier pass.

WHERE THIS DIFFERS FROM YOUR BRIEF

Your brief's own scoring favored the Flutter-style approach. We're not following that scoring here — not because we think the scoring itself was done carelessly, but because acting on it required the maintenance claims above, and those claims didn't hold up when we actually checked them.

6 Programming language & web framework

CHEAPEST TO REVERSE — GOT THE MOST DEBATE

DISAGREES WITH YOUR BRIEF'S OWN RANKING

WHAT WE RECOMMEND

Keep our original plan: one language (TypeScript/JavaScript) for both the server and the web application.

WHERE THIS DIFFERS FROM YOUR BRIEF

Your brief's own scoring ranked a Microsoft-technology combination (.NET on the server, paired with Microsoft's web framework, Blazor) as the top overall choice, with our recommended combination ranked near the bottom of the list it scored. We are not following that ranking, for two specific, structural reasons — not because we dispute the scoring exercise itself:

- **Microsoft's web framework can't serve arbitrary published websites.** Blazor is built to serve one style of web application well. Our platform needs to publish whatever kind of website any builder creates — which rules Blazor out regardless of how well the rest of .NET scores.
- **Because builders define their own data structures after the platform ships, a lot of .NET's biggest advantage doesn't apply where we need it most.** .NET's strongest selling point is that the compiler catches many mistakes automatically — but that only works when the shape of the data is known in advance. In our platform, the most complex and most changeable part of the system is data whose shape a builder defines at runtime, which that advantage can't reach.

To be fully transparent: on the brief's own scoring dimensions, taken individually, .NET wins most of them. We are not choosing it anyway, for the two structural reasons above — not because we think the comparison itself was unfair to our recommendation.

WHY WE'RE PUTTING THIS LAST, NOT FIRST

This is the decision that generated the most back-and-forth in review, and it's also the cheapest of the six to reverse later — it doesn't touch how customer data is stored, and it can be changed for a specific component later without redoing the whole system. We recommend spending less of your review time here and more on items 1 and 2 above, which are far more expensive to get wrong.

One more open question, not yet resolved with you

Your brief was written assuming a specific kind of business software — HR, finance, and inventory systems, plus mobile. Al ahaMatic itself is not one of those; it's the platform other people use to build software like that. It's still an open question whether your brief's advice was meant to apply to the platform we're building, or to the kinds of applications people will eventually build *using* the platform — and the two would call for different answers in places. We haven't settled this with you yet, and how much of this sheet applies as-is depends on the answer. We know you haven't yet had a chance to read through the full specification for the platform, and expect some of this may shift once you have.

Nothing in this document is final. Every recommendation above takes effect only once you approve it and it is recorded in our formal decision log.